B1.1 Approaches to computational thinking

How can we apply a computational solution to a real-world problem?

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- B1.1.1 Construct a problem specification
- B1.1.2 Describe the fundamental concepts of computational thinking
- B1.1.3 Explain how the fundamental concepts of computational thinking are used to approach and solve problems in Computer Science
- B1.1.4 Trace flowcharts for a range of programming algorithms

B1.1.1 Problem specification

Ever since their beginnings, computers have required a method to instruct them to perform a specific task. Now, we provide instructions to a computer via a programming language. Ada Lovelace, Charles Babbage, Alan Turing and Konrad Zuse are all recognized for their contributions to the development of coding and computer languages. Initially, programming languages were developed as a series of steps to wire a particular program. Then, they developed into a series of steps typed into a computer and then executed. Later, they acquired more advanced features, such as iterations; branching and even polymorphism; inheritance; and other object-oriented programming principles.

Even when tackling straightforward problems, it is essential to furnish the computer with precise instructions to enable it to carry out the tasks and resolve the problem.

However, you will not be able to provide clear instructions on how to solve a problem until you clearly outline the problem specifications.

A **problem specification** is a short, clear explanation of an issue, outlining who the **stakeholders** are and why it is important to solve the problem. The problem specification may include a problem statement; constraints and limitations; objectives and goals; input and output specifications; and evaluation criteria.

This is a great opportunity to think of your internal assessment project. When you define the **problem statement**, you need to include a description of the problem itself, who the solution is designed for, the issues encountered and what needs to be solved. To clearly understand the problem, you are encouraged to collect information from existing literature and research, use previous experiences with the problem and discuss it with multiple stakeholders who are impacted by the problem. In this way, you will be able to identify some possible constraints and limitations, for example:

- Limitations regarding the available technical requirements (hardware or software equipment)
- Economic aspect (cost of producing the solution)
- Legislation (regulations regarding the software development; ethical, social and legal aspects)
- Operational issues (workforce available)
- Schedule (time required to develop and implement the solution).

◆ **Problem specification:** a short, clear explanation of an issue, which may include: a problem statement; constraints and limitations; objectives and goals; input and output specifications; and evaluation criteria.

◆ **Stakeholder:** an individual or group(s) of people within or outside an organization who are affected or think they are affected by a software development project.

◆ **Problem statement:** a description of the problem itself, identification of who the solution is designed for, the issues encountered and what needs to be solved.

Once those are clearly defined, in collaboration with the main stakeholders you should outline the objectives and goals of the proposed solution, identifying what needs to be solved and what you want to achieve.

Every solution will include some form of input and output. Knowing how the input is being provided, which input is supplied and the expected outcome or output to be produced will help you understand the required process to reach your goal. The input can be in different forms:

- Direct entry (by using barcode scanners; OCR or OMR scanners; or MICR readers)
- Manual entry (keyboard, joystick, touch screen, touch pad or mouse entry, or data manually being entered by human operators)
- Automatic data entry (by using sensors: temperature, light, infrared, pressure, and so on).

Each of those has advantages and disadvantages. For example, manual entry might be cheaper, but it is prone to errors, while automatic entry is clearly more expensive due to the hardware or software involved, but it is more accurate and faster.

When it comes to output, this can be classified as temporary output (displaying the information on a screen), permanent output (printing the data), or electrical or mechanical output (using actuators: switches or relays).

Identifying the input data required and the output expected helps in outlining the data flow and understanding how the data travels through the proposed solution.

Evaluation criteria is the last step in constructing a problem specification. Criteria should be clear, specific, measurable and related to the functionality to be achieved through the proposed solution. This will allow you to use these criteria to evaluate the success of the product at a later stage.

Key information

Problem specification is part of the internal assessment requirements for criterion A. It is considered the starting point of the solution, and it must be used as a basis for the development of the product. The success criteria identified in the problem specification will be used in the planning, development and the evaluation of the product.

REVIEW QUESTIONS

1. Identify three stakeholders in a technical shop selling gaming consoles, games and IT equipment.
2. Define the term "stakeholder".
3. Define the term "problem specification".
4. State three possible constraints and limitations when considering developing a computational solution for a school.
5. In your school, identify those operations that have already been computerized; those that might be computerized soon; and those that are unlikely ever to be computerized.
6. For those activities you have identified as being already computerized at your school (for question 5), identify the inputs and outputs of the system.
7. Identify three reasons why there is a need to formulate a problem statement precisely.

Top tip!

Performance issues related to the lack of identifying limitations and constraints, and inputs and outputs specific to different systems in geographically diverse locations, may hinder end users and reduce compatibility between systems.

ACTIVITY

Imagine you must design and create an online platform to be used globally.

There are several constraints and limitations to take into consideration to ensure the platform is scalable, user friendly, efficient and accessible across different regions.

Discuss:

- language and regional differences, e.g. currencies, languages supported, date formats, units of measurement
- legal requirements, e.g. GDPR in Europe
- consumer-protection laws / content restrictions
- cultural differences and user behaviours that impact the design of the platform, e.g. meanings of colours for different cultures, sensitivity of specific content, user-interface alignment (left to right or right to left), time zones, scheduling.

How do such constraints support or limit the development of online platforms that can be used worldwide?

Is targeting a local market more advantageous and efficient, rather than targeting a global market?

B1.1.2 Fundamental concepts of computational thinking

Abstraction

◆ **Abstraction:** having a higher-level, simplified model to represent a complex system. It allows you to focus on the core ideas or concepts that matter, without being overly concerned about the intricate details of implementation.

Abstraction is the process of extracting essential information, while disregarding irrelevant data, to propose or outline a feasible solution to a given problem. In this way, simplified models can be designed; models that exclude unnecessary details. This plays a crucial role in providing a solution that satisfies the user requirements and needs, as it solves the problem without including unnecessary features, and in a shorter period due to the reduced amount of code written.

Real-world examples of abstraction include designing a map as a representation of a territory; a painting as a representation of a landscape; and a timetable. In programming, abstraction is an important concept in object-oriented programming. It is used to hide complexity from the user by:

- abstracting data entities (by hiding data entities via a data structure, reducing the body of the data to a simplified version of the whole)
- hiding underlying implementation of a process (programmers don't need to know details of how the subroutines are implemented, or what other subroutines they call, but they can simply use them to serve their purpose).

By using abstraction:

- the time required to create a piece of software is reduced
- the program becomes smaller in size, so it requires less space in memory and the download times are reduced
- customer satisfaction increases, as their requirements are met without extra features.

TOK

What counts as knowledge?

The map as an abstraction of the territory: A map is not the actual territory it represents, but rather a diagrammatical representation of an area, including some features and excluding others. The London Tube map was designed in 1933 as a simplified model of reality, informing the traveller how to navigate between stations, but excluding many other details and not providing an accurate representation of the actual space. Investigate and identify the differences between the London Tube map and other subway maps that you know.

Knowing the map doesn't mean that you know much about the actual territory, just as knowing the names of different items in different languages doesn't reflect your knowledge about the items themselves.

Watch Richard Feynman's "Names Don't Constitute Knowledge", or analyse the following quotation to further explore the concept:

> *Naming things is a human act; it is not an act of nature. We are the ones who, through language, create things out of the phenomena around us. Yet we forget that we control this process and let the process control us. Naming things – using language – is a very high-level abstraction, and when we name something we "freeze" it by placing it in a category and making a "thing" out of it. Language is a map, but three important things to remember about maps are: the map is not the territory; no map can represent all aspects of the territory; and every map reflects the mapmaker's point of view.*
>
> Lutz, Wiliam (1996) *The New Doublespeak: Why No One Knows What Anyone is Saying Anymore*. HarperCollins, New York, NY.

Investigate how knowing the name of something can positively or negatively influence our life experiences.

Algorithmic design

Before starting to write actual code, you should analyse and identify the requirements of the problem and then understand the logical steps required to solve the problem. Once you have a firm grasp of the requirements, the next step involves designing a potential solution. One effective approach for achieving this is to create an algorithm. This involves creating step-by-step solutions with predictable outcomes.

◆ **Algorithm:** a finite sequence of instructions that needs to be followed step-by-step to solve a problem.

An **algorithm** is a structured set of sequential instructions designed to address and resolve a problem.

Consider the following problem:

"A user is required to provide two whole numbers. Construct a program that calculates the sum of the two numbers and displays it."

The algorithm corresponding to the problem above is:

- **Step 1:** Ask the user to enter a number.
- **Step 2:** Store this number.
- **Step 3:** Ask the user to enter another number.
- **Step 4:** Store this new value.
- **Step 5:** Add the two numbers together.
- **Step 6:** Store the result.
- **Step 7:** Display the result.

Those steps need to be very specific and in the right order to be able to solve the problem. By applying algorithmic designs, you will develop algorithmic thinking skills that will help you develop efficient problem-solving techniques, by using structured and systematic algorithms.

Top tip!

When outlining algorithms, ensure the instructions are very specific, clear and in the right order. Not following the required order often leads to the wrong solution or different errors. Imagine you need to calculate the average of three numbers. Setting the value of sum to 0 after storing the total of the three values into the variable sum and attempting to divide this by 3 afterwards would produce an error.

Decomposition

◆ **Decomposition:** breaking down complex problems into smaller, more manageable parts.

◆ **Pattern recognition:** identifying similarities in the details of problems.

Decomposition refers to breaking down complex problems into smaller, more manageable parts. After designing solutions to those smaller problems, they can be put together to build up a final solution to the complex problem. This concept supports modularity, allowing multiple programmers or experts to collaborate and work simultaneously on solving the problem.

In programming, decomposition is often used to structure the solution, by designing several methods or functions.

Common mistake

Students do not always use terminology in an appropriate and competent way, and may approach questions by providing general superficial knowledge, which does not gain full marks.

Students often define "decomposition" as breaking down a program into smaller sub-programs. This isn't accurate as, at the stage decomposition occurs, there is no program created, therefore the problem is the one being broken down into smaller, more manageable parts.

Pattern recognition

Pattern recognition refers to identifying similarities in the details of problems. This simplifies the process of finding a solution by identifying patterns and focusing on reusing solutions proposed to solve those similarities. This means that you will develop reusable code in the form of functions or procedures; reuse existing code that has already been tested; and support the use of modularity, which reduces the development time.

B1.1.3 How fundamental concepts of computational thinking are used to approach and solve problems in Computer Science

◆ **Computational thinking:** a toolkit of available techniques for problem-solving; its fundamental concepts are abstraction, decomposition, algorithmic thinking and pattern recognition.

Computational thinking is not programming, and it does not make you think like a computer, but rather it makes you think like a computer scientist. It is a toolkit of available techniques for problem-solving. This gives you the skills to efficiently outline a problem specification; to analyse, understand and simplify the problem; and to identify and choose optimal solutions to different problems.

The fundamental concepts of computational thinking, such as abstraction, decomposition, algorithmic thinking and pattern recognition, can be used to solve real-world problems, for example: software development, data analysis, machine learning, database design and network-security problems.

In each of the areas identified above, all the fundamental concepts are equally important:

- **Software development:** You cannot create a program before:
  - understanding the problem
  - making abstraction of unnecessary details
  - finding repeating patterns
  - designing efficient algorithms

  Without any of these steps, the software produced might lack accuracy or might not be as efficient as it should be.
- **Games development:** Abstraction is used when the players are provided with a series of clues, some of which are intended to mislead the players. Abstraction refers to disregarding unnecessary details. Players should disregard such clues and focus on the important details.
- **Programming:** Programming languages offer libraries with functions and methods for programmers to use. The programmer makes an abstraction of the way those functions were written, focusing on correctly using them to complete their code.
- **Data analysis:** Computational thinking is used to automate repetitive tasks, predict market trends and improve customer service. Data analysts identify patterns (for example popular products for a category of people, repetitive tasks, frequent customer complaints) and apply algorithmic thinking to propose feasible solutions and break problems into simpler steps, saving hours of extra work on a weekly basis.
- **Machine learning:** Pattern recognition is an important concept, used in classifying data by finding patterns in large amounts of data, for example predicting purchasing behaviour based on buying habits. It can also be used to identify the skills required to be a good football player, by analysing video recordings to automatically find patterns in the behaviour of professional players. The same task might make use of abstraction to exclude irrelevant information provided by the videos and algorithms to promote those skills among new players during their virtual training sessions.
- **Database design:** Abstraction can be used to identify which data sources are relevant and which can be disregarded. Decomposition can be used to design relational databases by breaking down the complex problem into smaller ones. Entities can be represented as tables, and relations shown between them.
- **Database normalization:** Pattern recognition can be used to ensure there are no repeated groups of attributes or algorithmic design in outlining the tables' structures, and identify the logic behind the types of relationships established between tables.
- **Network security:** For solving network-security problems, abstraction enables the generalization of complex security models; decomposition is used to break down cybersecurity ecosystems into models that allow a clear identification of their security roles; pattern recognition is used to outline ways to identify and classify possible threats to the network; and algorithmic design is used to propose clear, step-by-step instructions on how to deal with such risks in similar situations.

TOK

Knowledge and AI

AI is rapidly improving, and it can easily achieve goals that people previously considered impossible. Machines can spot patterns at amazing rates, take decisions, output surprising results and even learn new things. But how do they get access to their wide range of data? What is the role of our digital footprint in improving machine learning techniques? And does online data about you give the full picture of what you are really like as a human being? Systems predicting human behaviour could lead to discrimination, so how do we ethically define the limits of knowledge that has been created with the help of technology?

Machine learning pushes the boundaries of how we perceive knowledge. People are excellent at generalizing, identifying patterns and predicting future actions or outcomes based on previous experiences. However, recent technological developments show that machine learning can compete with humans even in this area. Researchers are using symbolic and statistical AI to teach machines to reason about what they see. They can beat humans at a chess game, create original or fake art and provide medical diagnoses, and they can do all this with minimal or no human intervention. What does this mean for knowledge? Can a machine ever "know" something? Could knowledge reside outside human cognition?

To further explore the concept, try to create a deepfake video. How does this experience change your view on the saying "seeing is believing"? How does not knowing how to distinguish between good and bad knowledge influence our choices between the most appealing and the most accurate information?

ACTIVITY

Reflective: Consider how you achieve success and how you could change your approach when learning becomes challenging.

Social skills: Listen actively to other perspectives and ideas – there are different ways to solve a problem, some better than others; listen to advice and try new techniques and problem-solving strategies.

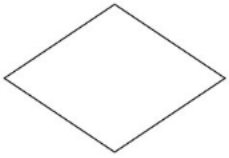
REVIEW QUESTIONS

1. Identify three examples of abstraction in Computer Science.
2. Define the term "decomposition".
3. Outline the algorithm for making a cup of tea.
4. Outline an area where computational thinking is used in Computer Science.
5. Define the term "algorithm".
6. Research the bubble sort algorithm. Outline the steps for this algorithm.
7. Research the swap puzzle activity and try to outline an algorithm to solve it in as few steps as possible.

B1.1.4 Flowcharts

Flowcharts are used to design algorithms, and to describe them using diagrams. They can be used to track variable changes, to show execution flow and to determine the expected output of an algorithm.

Standard flowchart symbols

Symbol	Name	Description
	Terminator	Start or end of the process
	Input / output	Input or output of data
	Process	Action, such as a calculation or an assignment
	Decision	True / false or yes / no decisions (selection statements)
	Flowline	Direction of data flow between shapes
	Connector	Continuation of a flow through multiple pages or charts

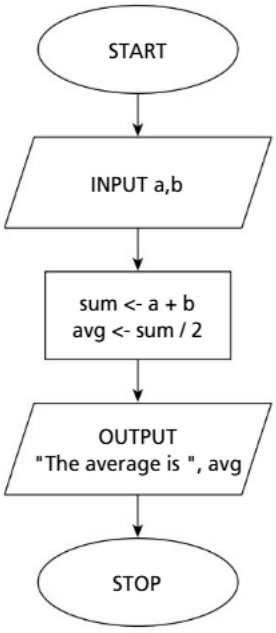
Consider the following problem:

Request the user to input two numbers from the keyboard. Output their average.

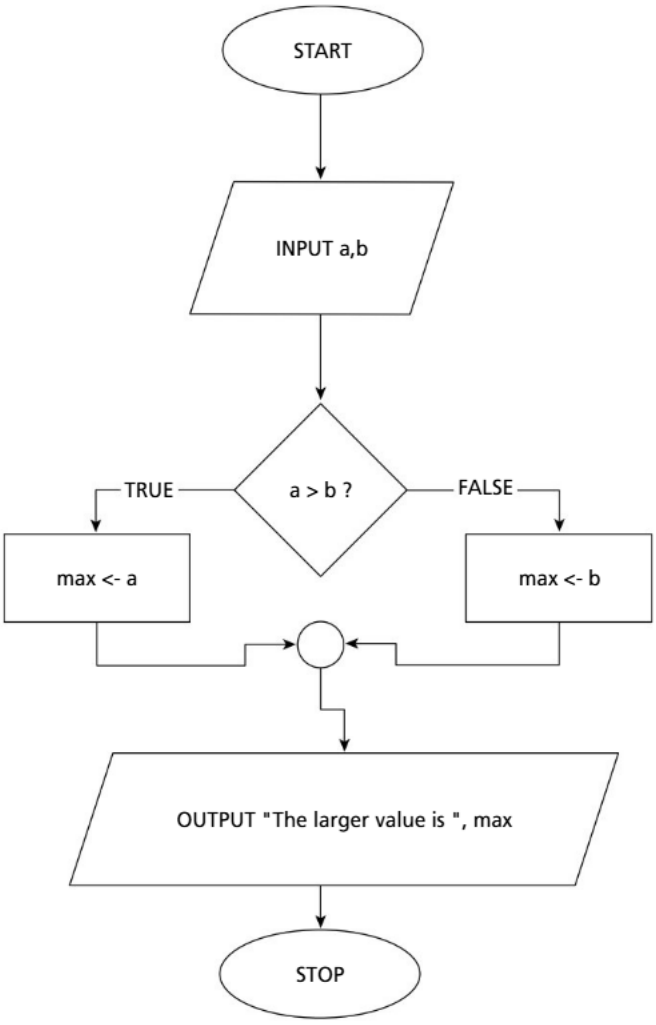
To solve the problem, identify the input, processes and output:

- **Input:** the two numbers (a, b)
- **Output:** the average of the two numbers (avg)
- **Processes:** calculate the sum, calculate the average.

The flowchart corresponding to the proposed solution is given below:



Flowcharts can become a little more complex by including selection or iteration. For example, the flowchart corresponding to an algorithm that outputs the larger of two different input numbers requires selection statements:



If you wanted to check the algorithm above you could test it with different test data, such as 7 and –3. To find the expected output, a table can be drawn and traced. The table includes the variable changes, decisions and outputs expected.

a	b	max	a>b	output

To trace the table and reach the final output, you need to go through the flowchart and follow the data flow shown by the arrows.

In this case, the first happening in the flowchart is the input. So, as a is the first input it will take the value 7 and b will be set to –3, and the table will look like this:

a	b	max	a>b	output
7	-3			

Common mistake

Students often forget to label the branches of decision boxes when drawing flowcharts. An unlabelled branch would not allow the examiner to identify which process is executed when the condition evaluates to True (Yes) and which executes when the condition evaluates to False (No). Also, make sure the flowlines are connected and none have no connection to a shape.

The next step is to check if the value stored in a is higher than the value stored in b.

a	b	max	a>b	output
7	-3			
			TRUE	
		7		

Finally, the output will be displayed.

a	b	max	a>b	output
7	-3			
			TRUE	
		7		
				The larger value is 7

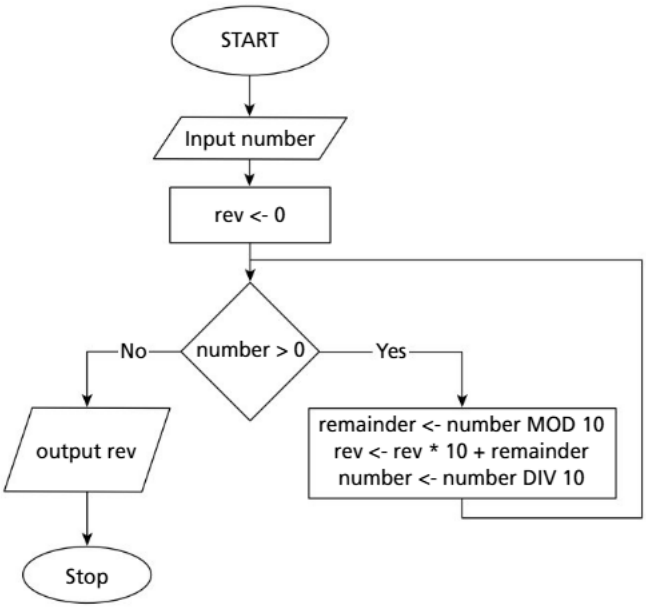
Please note that you don't have to insert each new value on a new line, but this was done just so you can notice the order of execution of the given operations. Trace tables will be further explored in B2 Programming.

REVIEW QUESTIONS

1 Draw a flowchart that would represent a solution for the following problem:
"Initialize a total to zero. Ask the user to enter 50 integer numbers, add the positive numbers to the total and count how many negative values were entered. Output the total and the count value."

2 Research the insertion sort algorithm. Draw a flowchart for this algorithm.

3 Consider the following flowchart.



a Research the role of the MOD and DIV functions. Trace the flowchart to find the output for the number 3452 and for the number 1760.

Input: 3452

number	rev	number > 0	remainder	output

Input: 1760

number	rev	number > 0	remainder	output

b Use pattern recognition to predict the output for the number 453453554651.

c Identify the purpose of the algorithm.

d Identify a problem with this algorithm.

Top tip!

When filling trace tables, ensure that you write the value a variable takes after an assignment, even if it is a repetition of the previous value. If the variable does not change for a portion of code, you can leave that section blank or rewrite the repeating values.

ACTIVITY

Thinking skills: Critical and creative thinking: A small family business that delivers goods within its small city is looking to further expand its reach. It is thinking of the following scenarios:

- **Creating brochures, which would include its products and its phone number, and distributing them in the three neighbouring cities.** It would take the orders by phone and deliver them as before, with cash payment on delivery.
- **Creating an online platform that would allow it to promote its products.** Customers would place the orders online and pay for their purchases online, and the company would deliver the goods via available transportation services within the country.
- **Promoting its business via social-media channels.** It would take the orders via instant-messaging services with bank-transfer payments, and deliver the products via transportation services available within the country.

Choose one of the scenarios above. Prepare a presentation that includes a problem specification. Identify the stakeholders; the problem statement; constraints and limitations; objectives and goals; input and output specifications; and evaluation criteria.

Consider the probable cost involved in implementing the scenario you have chosen, the time required to implement it, the hardware and software requirements, and possible effects on the community and staff members.

Deliver your presentation to the class and receive feedback from your peers.

Linking questions

1 How is pattern recognition used to identify different types of traffic flowing across a network? (A2)
2 How are the concepts of computational thinking used in code when designing algorithms? (B2)

EXAM PRACTICE QUESTIONS

Note: All the exam practice questions are representative of those that will be found on Paper 2 for the International Baccalaureate Diploma in Computer Science.

1 Define the term "computational thinking" and outline its role in problem solving. [3]
2 Outline the role computational thinking techniques like decomposition and abstraction play in software development. [4]
3 Identify three items that should be included in the problem specification and define one of them. [4]
4 Explain how pattern recognition can be used in data analysis, machine learning and database design. [6]
5 A teacher is asking 30 students how long they spend each day reading. The students will specify this duration in minutes and hours, for example 1 hour and 20 minutes. The teacher wants to write an algorithm that will output their input in minutes only.
   a Identify the input, process and output required for this algorithm. [3]
   b The teacher wants to create a ranking and send to parents the list of students in descending order based on their time spent reading books. Outline the steps required (the algorithm) to complete this task. [3]
   c Identify two stakeholders involved in this process. [2]
   d To keep their personal details anonymous, the teacher decides to create a username for each student. The username is made of the last two characters of their first name and the first three characters of their last name.
      i The first student's name is Sam Sung. State the corresponding username. [1]
      ii Draw a flowchart to outline the creation of usernames for the 30 students. [4]
      iii Explain one limitation of this algorithm and propose a better one. [3]